

Midterm

Exam Instructions

- The exam is designed to take 75 minutes.
- You are only allowed to refer to the course textbook and lecture slides during the exam. All other resources, including personal notes, online content, and AI tools, are strictly prohibited.
- You must clearly explain the process used to derive each answer. Answers without supporting explanations or derivations will not receive full credit.
- Show all intermediate work and derivations clearly. If the online exam does not provide space for this, use separate scratch paper and submit it in ELMS-Canvas (Canvas->Assignments->Midterm->[Supporting Work Upload](#)). Unreadable or missing supporting work will not be graded.
- You must complete the exam independently. Collaboration with others in any form is strictly prohibited.
- Do not share or discuss exam questions or content with anyone during or after the exam period.
- Any form of cheating or plagiarism will result in a zero on the exam and may lead to further disciplinary action.
- Your work must be entirely your own. Any content that closely resembles material generated by AI tools or submitted by another student, *even if reworded*, will be treated as a serious academic integrity violation. (Similarities that arise from using standard terminology or following class-taught methods are expected and not problematic.)

1

Multiple Choice 1 point

I pledge on my honor that I have not given or received any unauthorized assistance on this assignment/examination.

- ☐ I agree.
- ☐ I disagree.

You are given $f(n) = n \log n^3$. Which of the following statements correctly justifies whether or not $f(n) \in \Theta(n \log n)$? Select all correct statements. (Optional: briefly explain in your scratch paper why the others are incorrect for partial credit.)

- ☐ Observe that $n \log n^3 = 3n \log n$. Since we can ignore the constants in asymptotic notation, $\Theta(3n \log n) = \Theta(n \log n)$, so $f(n) \in \Theta(n \log n)$.
- ☐ We can use the Master Theorem to determine the asymptotic behavior of $f(n)$.
- ☐ Since $n \log n^3$ grows faster than $n \log n$, we conclude that $f(n) \notin \Theta(n \log n)$.
- ☐ If $\lim_{n \rightarrow \infty} \frac{n \log n}{n \log n^3} = 0$, then this proves $f(n) \in \Theta(n \log n)$.
- ☐ You can justify that $f(n) \in \Theta(n \log n)$ by showing that there exist positive constants c_1, c_2, n_0 so that $c_1 \cdot n \log n^3 \leq n \log n \leq c_2 n \log n^3, \forall n \geq n_0$.

Recurrence Relation

Suppose you are solving a puzzle game using divide-and-conquer approach. Consider the following pseudocode:

```
def solve_puzzle(P):
    n = size of P
    if n==1:
        return P

    subproblems = divide(P, m)      # Divide input n into m equal-size groups
    results = array of size m        # s will store solutions for each group

    for i = 0 to m-1:               # loop through 0 to m-1
        results[i] = solve_puzzle(subproblems[i])

    return combine(results)          # Combine the solutions from each group
```

The functions $divide(n)$ and $combine(s)$ have time complexities $\Theta(n)$ and $\Theta(n^2)$, respectively.

3 Essay 4 points

Write a recurrence relation to express the time complexity of this algorithm.

4 Essay 6 points

Let $m=2$.

1. Draw or describe the recursion tree that corresponds to the recurrence. You do not need to compute the full summation or solve the recurrence algebraically. Instead, focus on clearly describing the structure of the recursion tree, including: the number of nodes at each level, the amount of work done per level, when and how the recursion stops.

2. Briefly explain how the choice of m influences the number of levels and total accumulated work, which in turn determines the time complexity.

5

Essay 6 points

Let $m=2$. If the function `divide(P, m)` does not split the input into two equal-size groups, but instead groups them unevenly at each step (e.g., randomly). Assume that it doesn't affect the time complexity of the `combine()` function.

How would this affect the worst-case time complexity of the algorithm?

You do not need to solve the recurrence algebraically. Instead, focus on reasoning about how the depth and shape of the recursion tree might change in the worst case.

6

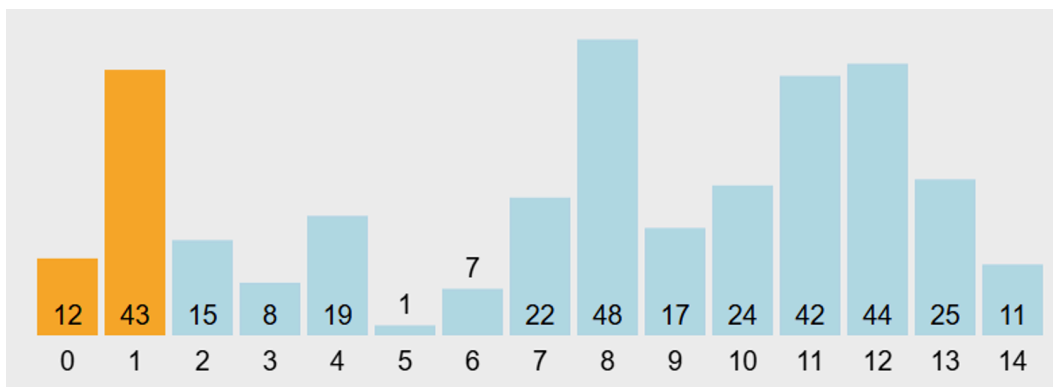
Multiple Answer 4 points

Which of the following statements about ADTs are correct? (Select all that apply.)

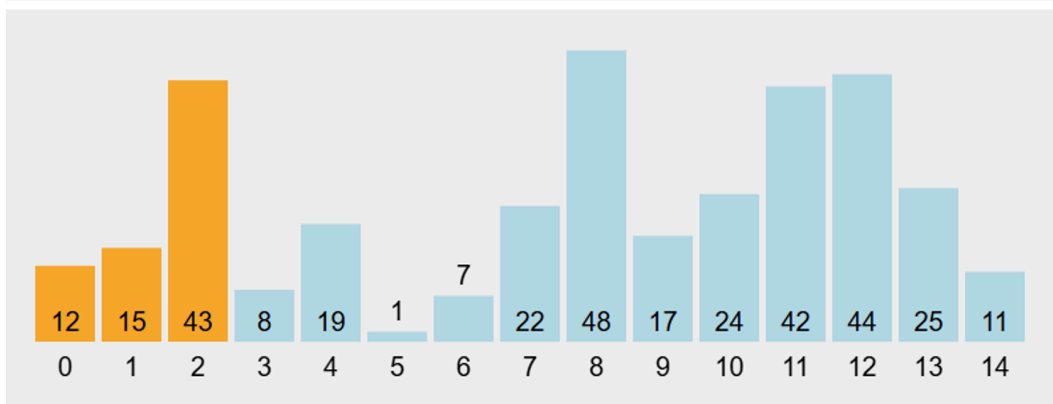
- ☐ The List ADT can dictate whether the list is implemented using a linked list or an array.
- ☐ The Queue ADT typically supports insertion and deletion at a random index.
- ☐ ADTs are theoretical concepts, while data structures are concrete implementation.
- ☐ An ADT can define edge cases and specify how they should be handled.
- ☐ Functional signatures tell you input and output types of each operation.

Basic Sorting Method

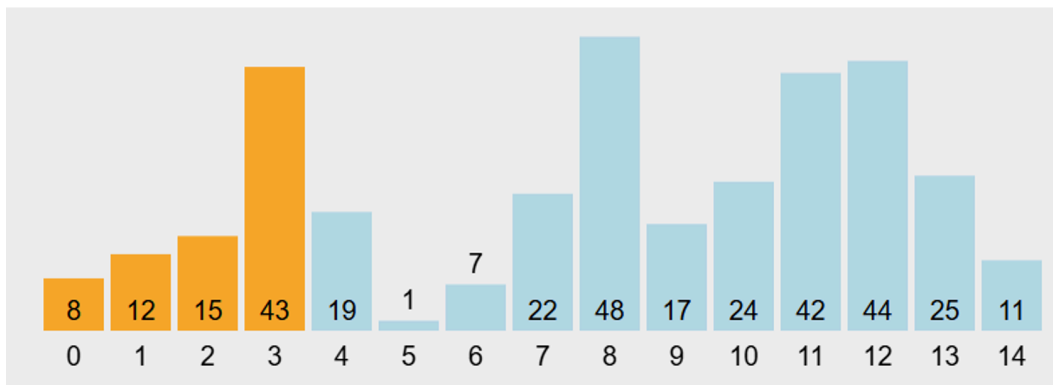
(1)



(2)



(3)



7

Essay 4 points

Given the current progression of the array, predict the sequence after one additional step.

```
students = [  
  {"name": "Alice", "dob": "2000-05-01", "uid": 402134},  
  {"name": "Bob", "dob": "2000-05-01", "uid": 200015},  
  {"name": "Bob", "dob": "1999-09-12", "uid": 300208},  
  {"name": "Alice", "dob": "2000-05-01", "uid": 401765},  
  {"name": "Alice", "dob": "1998-10-10", "uid": 501342},  
  {"name": "Charlie", "dob": "2001-03-03", "uid": 600017},  
  {"name": "Alice", "dob": "1998-10-10", "uid": 509871},  
  {"name": "Bob", "dob": "1999-09-12", "uid": 308624},  
  {"name": "Charlie", "dob": "2001-03-03", "uid": 608193},  
]
```

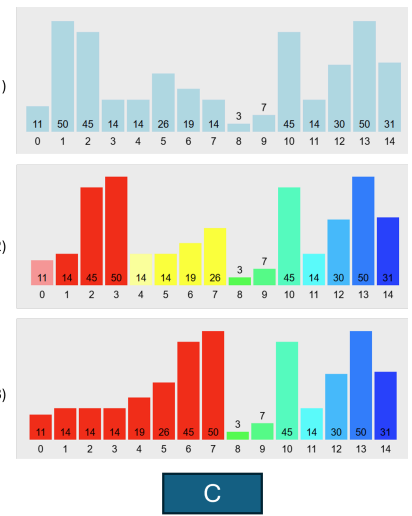
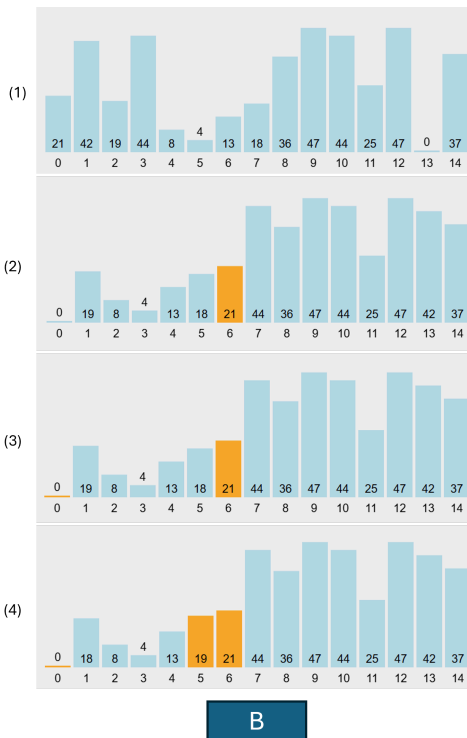
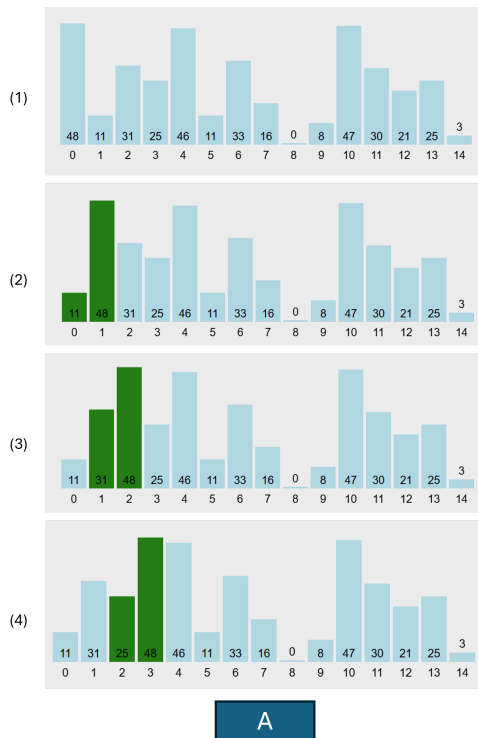
Your dataset consists of records with multiple fields (e.g., name, date of birth, UID, height). You want the final result to be sorted primarily by name; for records with the same name, by date of birth; and if those are also the same, by UID.

Using the particular sorting method shown above, you apply the following sequence:

1. First sort the data by UID.
2. Then, sort the output by date of birth.
3. Finally, sort the output by name.

Will this approach always yield the desired final order? Explain why or why not.

(Note: The image above is provided to help illustrate the records. You do not need to sort the data yourself.)



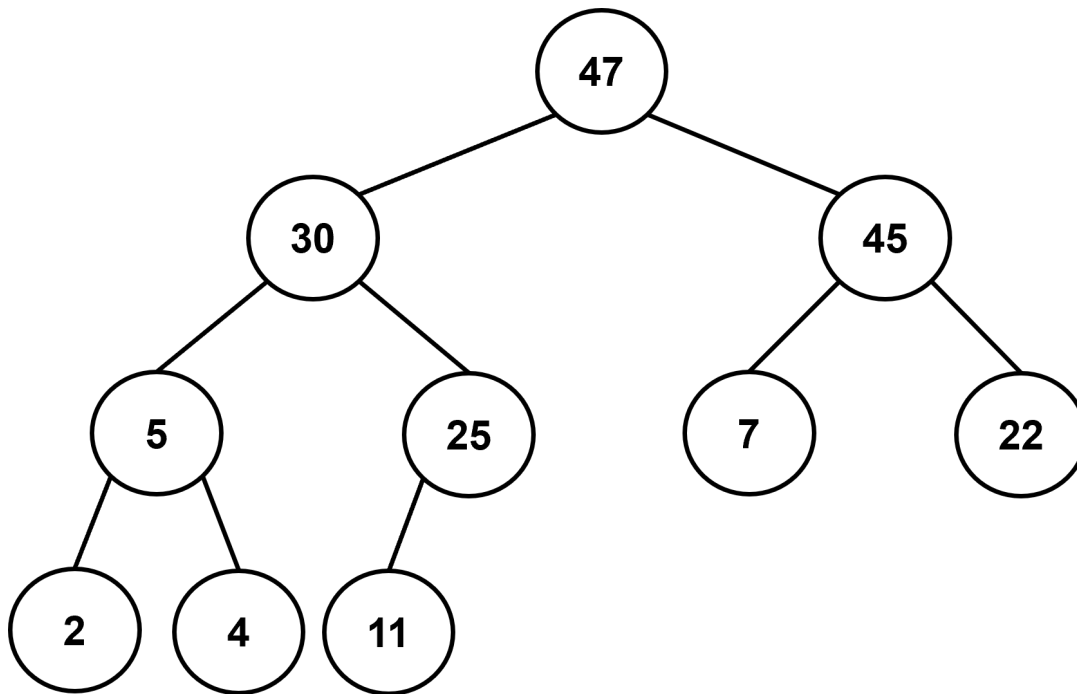
9 Essay 5 points

Suppose you are sorting two arrays:

- Array1 is already sorted in increasing order.
- Array2 is sorted in decreasing (reverse) order.

For each array, which sorting algorithm would you choose, and why? Explain your choices based on algorithmic behavior in best and worst-case scenarios.

Consider the following Heap:



1. Describe how the heap changes after inserting 33.
2. Based on the tree resulting from the previous insertions, describe how the tree changes after performing *Extract-Max*.

In the text box below, show the array representation of the Heap after each operation. Use your scratch paper to briefly show intermediate steps.

Your Binary Search Tree stores data using the following `TreeNode` structure:

```
TreeNode {
    data: Elt
    left: TreeNode
    right: TreeNode
}
```

The BST supports the following operations, defined by the ADT:

ADT: BST of Elt (Object-Oriented Signature)

- | | | |
|-------------|---------------------------------|--|
| • new: | -->BST | # Creates a new empty BST |
| • insert: | Elt -->Boolean | # Insert an element |
| • remove: | Elt -->Boolean | # Remove an element |
| • findMin: | -->Elt | # Return the smallest element |
| • findMax: | -->Elt | # Return the largest element |
| • contains: | Elt -->Boolean (searching) | # Check if an element exists |
| • get: | Elt -->BST (return a Node) | # Return the node containing the element |
| • val: | -->Elt (get value) | # Get the value stored at a node |
| • size: | -->Int+ (non-negative integers) | # Return the number of elements |
| • empty: | -->Boolean | # Check if the BST is empty |
| • height: | -->Int+ | # Return the height of the tree |

Write the pseudocode for a function

```
def val2(k: Elt) --> Elt:
```

```
    # TODO
```

```
    # if k is in the tree, return the smallest element that appears in the subtree rooted at
    the node containing k
```

```
    # if k is not in the tree, return null or raise an error.
```

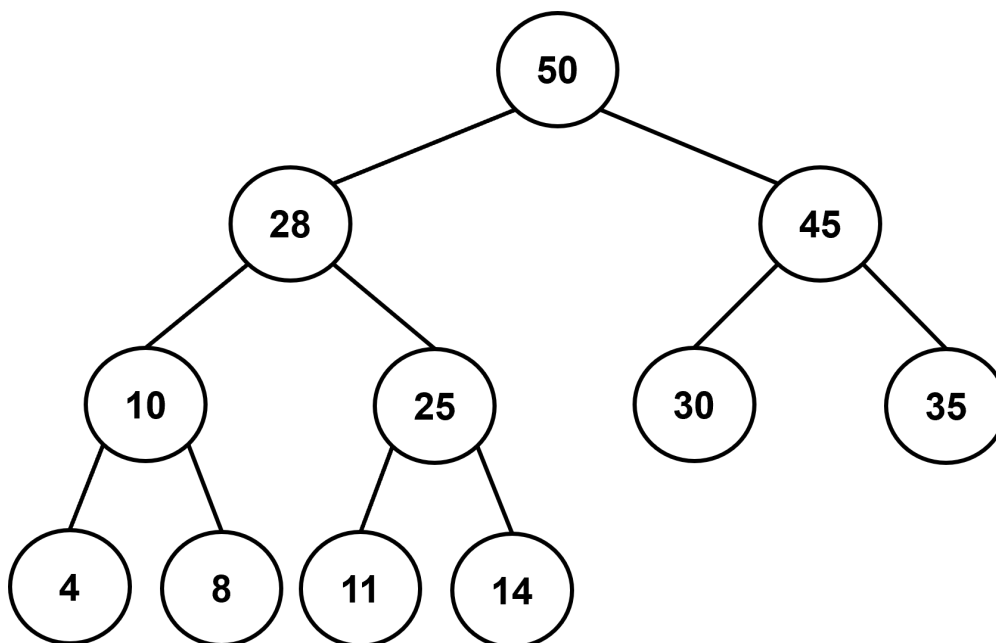
This function should:

- if k is in the tree, return the smallest element that appears in the subtree rooted at the node containing k
- if k is not in the tree, return null or raise an error.

You may only use the basic operations defined in the ADT above. Discuss the time complexity.

12 Multiple Answer 4 points

Consider the following tree:



Select all correct statements. (Based on the specific tree structure shown above and the data arrangement given.)

- ☐ A post-order traversal of this tree yields a sorted array.
- ☐ We can search for a key in this tree in $\Theta(n \log n)$ time, where n is the number of nodes.
- ☐ This tree is a valid binary search tree *and* also a valid binary heap.
- ☐ This tree has the shortest possible height for the given number of nodes
- ☐ This tree is a complete binary tree.
- ☐ Array representation is [39, 28, 19, 18, 22, 13, 17, 9, 14, 6, 21]

13 Essay 4 points

[True/False] Some NP problems can be solved in polynomial time.

14 Essay 5 points

Explain why decision problems can be classified as NP-complete, while their corresponding optimization problems are usually considered NP-hard.